

UNIT 2, BUILDING INTELLIGENT AGENT WORKFLOWS

LECTURE 5

Context-Aware Agents and Workflow Testing

The last of Unit 2. Two things: an agent that remembers what you said a moment ago, and the first honest look at how you test a system whose answers change every time you run it.

SECTION 00

Closing the unit

You can build, chain, and reach out. Two things left before you can trust it.

RECALL

The whole unit, so far

L1 **Chose the shape** Workflow, router, agent. The simplest that works wins.

L2 **Survived failure** A broken tool becomes a readable observation, not a crash.

L3 **Reached real APIs** The wrapper hides a messy service behind a clean tool.

L4 **Chained the steps** A pipeline that stops cleanly, and events that react.

THE TWO THINGS LEFT

A system you cannot hold a conversation with is a lookup, not an agent. And a system you cannot test is one you cannot trust in front of a user. Today closes both gaps, and with them, the unit.

Today

- Why every request so far has started from nothing, and why that is a problem.
- Session memory: the running transcript, and the facts you pin so they survive.
- How an agent resolves "make it urgent" into "make the billing ticket urgent".
- Why these systems are genuinely hard to test, and why the obvious approach fails.
- Invariant testing: how to check a system whose exact answer you cannot predict.

HOLD ON TO THIS

Both halves use the triage domain one last time. First a conversation about a ticket, then a way to test that the conversation always produces a valid result even though the wording changes every run.

SECTION 01

An agent that remembers

Conversation is impossible without memory.

The problem with a memoryless agent

User: route ticket 12 to billing

Agent: Routed to the billing team.

User: actually, make it urgent

Agent: Make WHAT urgent? I have never spoken to you.

THE GOLDFISH, FROM UNIT 1

Remember, the model forgets everything between calls. On its own it is a goldfish. Every turn is a stranger walking in mid-conversation. If the second turn is going to understand "it", something outside the model has to remember what "it" refers to. That something is a session.

A session: two kinds of memory

THE TRANSCRIPT

What was said, in order

The running list of turns. Lets the agent see the flow of the conversation. It is bounded: past a limit, the oldest turns drop, because from Unit 1 you know an unbounded transcript is a cost bomb.

THE PINNED FACTS

What must not be forgotten

A small set of named facts, kept separate. The chosen queue, the ticket id, the priority. These survive even when the transcript is trimmed, because losing them would break the conversation.

WHY TWO KINDS AND NOT ONE

If you keep only the transcript and trim it, you eventually trim away the one fact you needed. If you keep only facts, you lose the conversational flow. The split, a bounded transcript plus a handful of durable facts, is exactly the pinning idea from Unit 1 Lecture 3, now given a home.

Resolving the reference

```
User: route ticket 12 to billing
      -> session.remember("queue", "billing")

User: make it urgent
      -> "it" is a reference. look in the session.
      -> queue = session.recall("queue")    # "billing"
Agent: Marked the billing ticket urgent.
```

AND THE HONEST FAILURE

What if the user says "make it urgent" as their very first message, with no earlier ticket? The agent must not invent one. It says "I do not have an earlier ticket to refer to, which one do you mean?". A context-aware agent that guesses at missing context is worse than one with no memory at all.

SECTION 02

Testing the untestable

The same input can give a different answer. Now test that.

Why these systems resist testing

```
assert agent.reply("weather?") == "It is 31C in Mumbai."
```

This test is broken before you run it. The model might say "Mumbai is 31 degrees", or "It is currently 31C in Mumbai with light rain", or a dozen other true things. All correct. All different. Your equals check fails on every one of them.

THE CORE DIFFICULTY OF THE WHOLE FIELD

A traditional program is deterministic: same input, same output, easy to assert on. A system with a model in it is not. The same question gives different words every time. If you test it the old way, asserting exact output, every test is flaky, and a flaky test is worse than no test, because people learn to ignore it.

Test what must always be true

DO NOT TEST

The exact words

"It is 31C in Mumbai." The model will phrase it differently every run, so any exact-match assertion is doomed to flake. Stop testing the wording.

DO TEST

The invariants

The things that must hold no matter how it is worded: the queue is one of the five valid queues, the priority is never empty, the required field is present. These are stable even when the words are not.

AN INVARIANT IS A PROMISE THE OUTPUT MUST KEEP

You cannot promise the model will say a specific sentence. You can promise that whatever it says, the queue it picked is real, the priority it set is valid, and nothing required is missing. Test the promises, not the prose. This is how the whole industry tests systems with models in them.

Invariants in code

```
result = agent.triage(ticket)      # wording varies

checks = check_invariants(result, {
    "queue is valid":    queue_is_valid(),
    "priority present":  field_present("priority"),
    "priority allowed":  field_in("priority",
                                ["normal", "urgent"]),
})
assert all_passed(checks)          # stable across every run
```

THIS TEST NEVER FLAKES

Run it a thousand times, with the model wording the answer a thousand different ways, and it passes every time the answer is valid and fails every time it is not. That is a test you can trust, put in CI, and rely on to catch a real regression. It is the run log from last lecture grown into a discipline.

The three kinds of test you actually write

THE DETERMINISTIC PARTS

Your validators, your wrappers, your pipeline logic. These have no model in them, so test them normally, with exact assertions. Most of your code is this.

THE INVARIANTS

For the parts with a model, assert the promises that must always hold. Valid queue, present fields, sane ranges. Stable across every run.

THE FAILURE PATHS

From Lecture 3, test what happens when a tool or API fails, using a fake so you never touch the real thing. Most people skip this. It is where the marks and the incidents both live.

HOLD ON TO THIS

Notice most of your testing is ordinary. The model touches a smaller part of the system than students expect, and everything around it tests the normal way. Isolate the model, and most of your code becomes testable again.

Hold on to this

Memory makes conversation possible. It also makes testing hard. Both are today.

A session is memory you build

The model forgets. The session remembers, within a bound.

Pin what must survive trimming

A bounded transcript plus a few durable facts. From U1 L3.

Do not test the wording

It changes every run. Exact-match tests flake and get ignored.

Test the invariants instead

The promises the output must keep. Those never flake.

Before next session

DO

Break the memory bound

Set a session max of 3 turns, then have a 6 turn conversation that refers back to turn 1. Watch it fail, then pin the fact instead and watch it survive. Feel why pinning exists.

DO

Write an invariant test

For your Practical 2 agent, write one test that asserts an invariant, not exact output. Run it several times. Confirm it passes on valid answers regardless of wording.

THINK

What must always be true

List three invariants your capstone system must never violate. These become your test suite and, not coincidentally, your safety argument in Unit 6.

ON THE THIRD ONE

The invariants you list are the beginning of your Unit 6 safety work. "The agent never routes to a queue that does not exist" is a test today and a security guarantee later. The same idea wears two hats, and writing them down now pays twice.

End of Unit 2

Unit 3, Agent Development with Python and Frameworks

You can now choose an agent's shape, survive its failures, connect it to real services, chain its steps, give it memory, and test it. You built all of that by hand, so you understand every piece. Unit 3 hands you the frameworks the industry uses, Semantic Kernel and AutoGen, and because you built the ideas yourself first, you will see them for what they are: convenient packaging of things you already understand, not magic.

COME WITH

Your memory-bound experiment, your invariant test, and three invariants your capstone must never break.